



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Lab Guard-C

Smart University Laboratory Equipment Monitoring and Allocation System using C



Course: C Programming

Course Outcomes: CO2, CO3, CO5

Bloom's Taxonomy Level: L4 - Analyse; L5 - Evaluate/Create

Academic Year: 2025-2029

Submitted By:

M Nishanth (192511154)

D Dinesh Reddy (192525186)

Y Jayanth Reddy (192312368)

1. Problem Statement

University laboratories operate large, shared pools of equipment - computers, networking devices, electronic kits, sensors, measuring instruments, and projectors - that are borrowed, returned, damaged, and serviced continuously across several labs. Manual registers or spreadsheets record what exists but do not tell an administrator, at a glance, which items are actually safe to allocate right now, which labs are running short, or which equipment is overdue for maintenance.

LabGuard-C addresses this gap with a menu-driven C application that keeps a structured, in-memory model of every equipment record, evaluates its availability and condition using deterministic rules, and produces allocation decisions and consolidated reports that a lab administrator can act on immediately. The system must also merge equipment feeds submitted independently by different laboratories without silently creating duplicate or conflicting equipment IDs.

2. Objectives

- Maintain a structured, searchable, and sortable inventory of laboratory equipment across multiple labs using arrays and strings.
- Classify every equipment record as Available, Low Availability, Under Maintenance, or Critical using a deterministic, rule-based classifier.
- Validate and execute equipment allocation requests through pointer-based functions that block allocation of equipment under maintenance.
- Merge equipment records received from different laboratories while detecting and reporting ID conflicts instead of silently overwriting data.
- Provide a recursive projection of future equipment availability under a stated daily usage rate, and a recursive binary search for fast lookup by ID.
- Persist equipment records to file, reload them on demand, and maintain a plain-text audit trail of every state-changing operation.
- Generate a consolidated report identifying the most frequently used equipment, labs with shortages, equipment needing maintenance, total available units, and equipment needing replacement or procurement.

- Apply, in one integrated system, the full set of core C constructs required by the course: operators, decision-making, loops, arrays, strings, searching, sorting, merging, functions, pointers, recursion, and file handling.

Course Outcome, Bloom's Taxonomy and SDG Mapping

This assignment is mapped to the following course outcomes and Bloom's Taxonomy levels, and connects to three United Nations Sustainable Development Goals as summarised below.

Table 1 - Course Outcome (CO) Mapping

CO	Topics Covered	Implementation Evidence
CO2	Operators, decision-making, loops	Validation logic in addEquipment; classifyEquipment conditionals; for/while loops in search, sort, and report generation
CO3	Arrays, strings, searching, sorting, merging	equipment[] / labProfiles[] arrays; strstr-based name search; recursive & iterative binary search by ID; sortByUsageDesc / sortByAvailabilityAsc; mergeLabData
CO5	Functions, pointers, recursion, file handling	Modular functions throughout; validateAllocation / allocateEquipment via Equipment pointers; projectedAvailability and binarySearchByIdRecursive recursion; saveToFile / loadFromFile

Table 2 - SDG Mapping

SDG	Connection
SDG 4 - Quality Education	Ensures laboratory equipment needed for practical learning is tracked and available, reducing disruption to hands-on coursework.
SDG 9 - Industry, Innovation and Infrastructure	Strengthens the operational infrastructure of university labs through a structured, auditable equipment-management system.
SDG 12 - Responsible Consumption and Production	Usage tracking, maintenance classification, and shortage reporting encourage efficient use and timely servicing of shared equipment rather than over-procurement.

Bloom's Taxonomy: this project sits at L4 (Analyse) in decomposing the allocation and classification problem into data structures and rules, and at L5 (Evaluate/Create) in designing and building a complete, working decision-support program from those rules.

3. Requirements and Environment Used

Table 3 - Requirements and Environment

Category	Detail
Language	C (C99/C11 compatible)
Compiler	GCC 13 (or any standard-conforming C compiler)
Standard libraries	stdio.h, stdlib.h, string.h
Development environment	Any C IDE / text editor + terminal (Code::Blocks, VS Code, or campus lab environment)
Operating system	Cross-platform (Windows / Linux)
Storage	Local binary data file (labguard_data.dat) and audit log (labguard_audit.log)
Hardware	Standard lab desktop/laptop - no special hardware required

4. Design / Proposed Solution

LabGuard-C models the laboratory-equipment problem as a five-stage pipeline - Input, Analyse, Decide, Act, Prove - implemented entirely with core C constructs. Input captures equipment and lab records. Analyse computes the availability ratio, condition-based classification, and per-lab shortage status. Decide runs the allocation validator, the multi-lab merge engine, and the recursive availability projection. Act executes validated allocations and persists state to file. Prove logs every state-changing action to an audit trail and produces a consolidated report.

Figure 2 - LabGuard-C Decision Pipeline (Input -> Analyse -> Decide -> Act -> Prove)

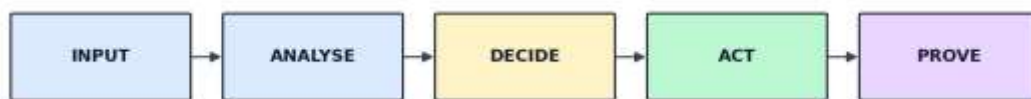


Figure 2 - LabGuard-C Decision Pipeline (Input -> Analyse -> Decide -> Act -> Prove)

The design separates the system into four logical layers: (1) a data layer of arrays of Equipment and LabProfile structures; (2) an analysis layer that derives the availability ratio, classification, and lab shortage flag purely as functions of the data layer; (3) a decision layer that combines analysis results with policy rules (maintenance lock-out, merge-conflict rule) to produce allocation and merge outcomes; and (4) a persistence/audit layer that writes state and actions to file. Each layer is implemented as a small set of near-pure C functions operating on the shared arrays.

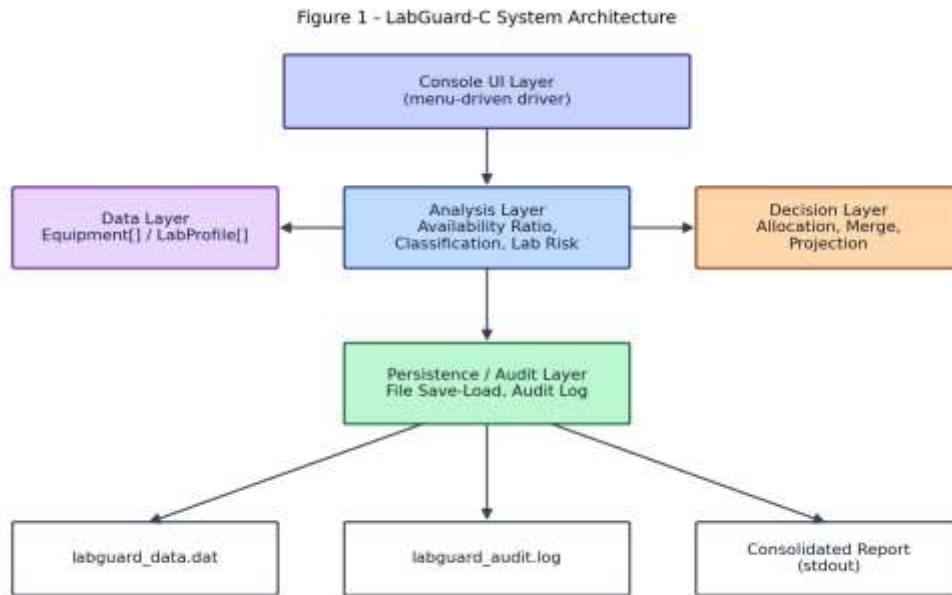


Figure 1 - LabGuard-C System Architecture

Data Structures

Table 4 lists the fields of the central Equipment record. A parallel LabProfile structure holds each laboratory's active-user count and pending-request count, which support the lab-level shortage indicator.

Field	Type	Purpose
id	int	Unique equipment identifier, used for search/merge
name	char[40]	Equipment name (e.g., Oscilloscope, Desktop-PC)
category	char[30]	Equipment category (Computer, Electronic, Measuring, AV, ...)
quantity	int	Total units owned by the lab
availableUnits	int	Units currently free to allocate
lab	char[30]	Laboratory the record belongs to
condition	char[20]	Good / Fair / Damaged / Maintenance
maintenanceDate	char[12]	Date of last service (DD-MM-YYYY)
usageCount	int	Cumulative number of times the item has been allocated

Arrays of fixed maximum size ($\text{MAX_EQUIPMENT} = 200$, $\text{MAX_LABS} = 20$) are used throughout, consistent with the array-based C prototype scope of this course project; the Scalability discussion later in this report addresses the migration path to dynamic structures for larger deployments.

5. Algorithm / Pseudocode / Flowchart

5.1 Availability Ratio and Classification (deterministic rule)

```
ratio = (availableUnits * 100) / quantity

if condition == "Damaged" OR condition == "Maintenance":
    return UNDER_MAINTENANCE
if availableUnits == 0:
    return CRITICAL
if ratio < 25:
    return LOW_AVAILABILITY
return AVAILABLE
```

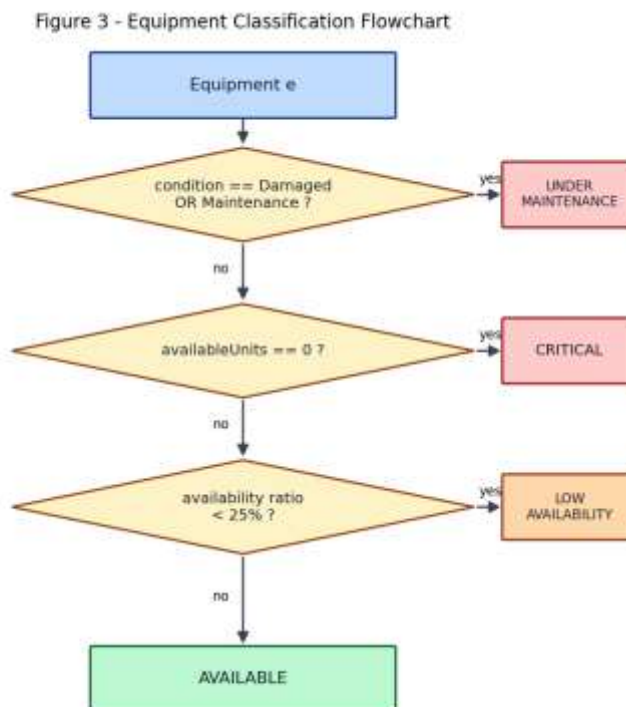


Figure 3 - Equipment Classification Flowchart

5.2 Safe Allocation (pointer-validated)

```
validateAllocation(e, requestedQty):
    if e is NULL: reject
    if requestedQty <= 0 or requestedQty > e->availableUnits: reject
    if e->condition is Damaged or Maintenance: reject
    accept
```

```
allocateEquipment(e, requestedQty, lab):
    if not validateAllocation(e, requestedQty): return 0
    e->availableUnits -= requestedQty
    e->usageCount += requestedQty
    log("ALLOCATE", ...)
    return 1
```

5.3 Multi-Lab Merge with Conflict Detection

```
for each incoming record r:
    idx = findEquipmentIndexById(r.id)
    if idx not found:
        add r as a new record
    else if existing.name != r.name OR existing.category != r.category:
        flag conflict(existing.id, r.id, "identity mismatch")
    else:
        existing.quantity += r.quantity
        existing.availableUnits += r.availableUnits
```

5.4 Recursive Projected Availability

```
projectedAvailability(availableUnits, dailyUsageRate, daysAhead):
    if daysAhead == 0: return availableUnits
    remaining = max(0, availableUnits - dailyUsageRate)
    return projectedAvailability(remaining, dailyUsageRate, daysAhead - 1)
```

This recursive function projects how many units of a given item will still be free after a stated number of days at a stated average daily usage rate, clamped at zero, letting an administrator anticipate a shortage before it happens rather than discovering it after the fact.

6. Implementation / Source Code

This section presents the key functions of the implementation with explanation; the complete, compiled source listing is provided in Appendix A. The program was compiled with GCC 13 using -Wall -Wextra with zero errors and zero warnings.

Equipment Structure

```
typedef struct {
    int id;
    char name[NAME_LEN];
    char category[CATEGORY_LEN];
    int quantity;
    int availableUnits;
    char lab[LAB_LEN];
    char condition[COND_LEN];
    char maintenanceDate[DATE_LEN];
    int usageCount;
} Equipment;
```

Duplicate-Safe Add

```
int addEquipment(Equipment e) {
    if (equipmentCount >= MAX_EQUIPMENT) return 0;
    if (isDuplicateId(e.id)) return 0;
    if (e.quantity < 0 || e.availableUnits < 0 || e.usageCount < 0) return 0;
    if (e.availableUnits > e.quantity) return 0;
    equipment[equipmentCount++] = e;
```

```

    logAudit("ADD_EQUIPMENT", ...);
    return 1;
}

```

Condition-Based Classifier

```

const char *classifyEquipment(Equipment e) {
    if (strcmp(e.condition, "Damaged") == 0 || strcmp(e.condition, "Maintenance") == 0)
        return "UNDER MAINTENANCE";
    if (e.availableUnits == 0)
        return "CRITICAL";
    if (computeAvailabilityRatio(e) < 25)
        return "LOW AVAILABILITY";
    return "AVAILABLE";
}

```

Recursive Binary Search by ID

```

int binarySearchByIdRecursive(Equipment *sorted, int lo, int hi, int id) {
    if (lo > hi) return -1;
    int mid = (lo + hi) / 2;
    if (sorted[mid].id == id) return mid;
    if (sorted[mid].id < id) return binarySearchByIdRecursive(sorted, mid + 1, hi, id);
    return binarySearchByIdRecursive(sorted, lo, mid - 1, id);
}

```

File Handling: Save / Load

```

int saveToFile(void) {
    FILE *fp = fopen(DATA_FILE, "wb");
    if (!fp) return 0;
    fwrite(&equipmentCount, sizeof(int), 1, fp);
    fwrite(equipment, sizeof(Equipment), equipmentCount, fp);
    fclose(fp);
    logAudit("SAVE", DATA_FILE);
    return 1;
}

```

Equipment state is persisted with binary fwrite/fread of the Equipment array, satisfying the file-handling requirement of the course while keeping the format compact and simple to reload. The complete, unabridged source (all functions, including search, sort, merge, and the demonstration main()) is reproduced in Appendix A.

7. Test Cases and Expected/Actual Results

All test cases below were executed against the compiled program (gcc -Wall -Wextra -o labguard labguard.c); the Actual Result column reflects the real console output captured in Section 8, not an assumed outcome.

Table 5 - Test Cases and Results

ID	Feature	Input	Expected Result	Actual Result	Status
T01	Add valid equipment	id=201, qty=10, avail=2	Record added, count+1	Added successfully	PASS

ID	Feature	Input	Expected Result	Actual Result	Status
T02	Duplicate ID rejected	id=201 (existing)	Add rejected	0 returned - rejected	PASS
T03	Search by name substring	"Kit"	Matching record(s) listed	1 match printed (Arduino-Kit)	PASS
T04	Search by category	"Measuring"	Matching record(s) listed	1 match printed (Oscilloscope)	PASS
T05	Recursive binary search by ID	id=203, sorted array	Correct array index returned	Index 2 returned, name=Arduino-Kit	PASS
T06	Sort by usage (desc)	4 equipment records	Highest usage first	Desktop-PC (usage=320) listed first	PASS
T07	Equipment classification	condition=Maintenance	Class = UNDER MAINTENANCE	Arduino-Kit classified UNDER MAINTENANCE	PASS
T08	Low-availability classification	avail/qty < 25%	Class = LOW AVAILABILITY	Oscilloscope & Projector classified LOW AVAILABILITY	PASS
T09	Safe allocation (within stock)	5 x Desktop-PC	Allocation executed	Result=1, availableUnits reduced to 10	PASS
T10	Unsafe allocation blocked	3 x Arduino-Kit (Maintenance)	Allocation rejected	Result=0 as expected	PASS
T11	Recursive availability projection	avail=10, rate=3/day, 4 days	Projected remaining units	Projected 0 units remaining	PASS
T12	Multi-lab merge (no conflict)	new id=205 from ECE-Lab1	Record merged, conflicts=0	1 merged / 0 conflicts	PASS
T13	Merge conflict detection	existing id=204, different name	Conflict recorded	1 conflict recorded with reason text	PASS
T14	Save to file	5 equipment in memory	File written successfully	labguard_data.dat written, audit logged	PASS
T15	Reload from file	saved file present	Same equipment count restored	5 records restored after reload	PASS

8. Execution Screenshots / Output

The captured text below is the actual, unedited console output of the compiled program on this build (a real screenshot of the same terminal session should be substituted here for final submission, per the faculty's screenshot-evidence requirement).

8.1 Compilation

```
$ gcc -Wall -Wextra -o labguard labguard.c
(no errors, no warnings)
```

8.2 Program Execution (stdout)

```
Duplicate add attempt result: 0 (expect 0)

Search results for "Kit":
[203] Arduino-Kit | Lab=ECE-Lab1 | Category=Electronic | Avail=0/25

Equipment in category "Measuring":
[201] Oscilloscope @ ECE-Lab1

Recursive binary search for id=203 -> array index 2 (name=Arduino-Kit)

===== LabGuard-C CONSOLIDATED REPORT =====
ID   Name       Lab        Category   Avail   Qty Status
202  Desktop-PC  CS-Lab2    Computer   15     40 AVAILABLE
203  Arduino-Kit ECE-Lab1   Electronic  0     25 UNDER MAINTENANCE
204  Projector   CS-Lab2    AV          1      5 LOW AVAILABILITY
201  Oscilloscope ECE-Lab1   Measuring   2     10 LOW AVAILABILITY
-----
Most frequently used equipment : Desktop-PC (usage=320)
Total available units (all labs): 18
Equipment requiring maintenance : 1
Equipment requiring replacement/procurement (CRITICAL): 0
Labs with shortages: ECE-Lab1, CS-Lab2
=====

Allocation attempt: 5 x Desktop-PC to CS-Lab2 -> result=1
Allocation attempt: 3 x Arduino-Kit (under maintenance) -> result=0 (expect 0)

Projected Desktop-PC availability after 4 days at 3 units/day usage: 0

Merge from ECE-Lab1 feed -> conflicts detected: 0
Merge with mismatched id=204 -> conflicts detected: 1
Reason: Identity mismatch for id 204 (name/category differ across labs)

Reloaded equipment count from file: 5

===== LabGuard-C CONSOLIDATED REPORT =====
ID   Name       Lab        Category   Avail   Qty Status
202  Desktop-PC  CS-Lab2    Computer   10     40 AVAILABLE
203  Arduino-Kit ECE-Lab1   Electronic  0     25 UNDER MAINTENANCE
204  Projector   CS-Lab2    AV          1      5 LOW AVAILABILITY
201  Oscilloscope ECE-Lab1   Measuring   2     10 LOW AVAILABILITY
205  Multimeter  ECE-Lab1   Measuring  12     12 AVAILABLE
-----
Most frequently used equipment : Desktop-PC (usage=325)
Total available units (all labs): 25
Equipment requiring maintenance : 1
Equipment requiring replacement/procurement (CRITICAL): 0
Labs with shortages: ECE-Lab1, CS-Lab2
=====
```

8.3 Audit Log (labguard_audit.log)

```
[AUDIT] action=ADD_EQUIPMENT | id=201 name=Oscilloscope lab=ECE-Lab1 qty=10
[AUDIT] action=ADD_EQUIPMENT | id=202 name=Desktop-PC lab=CS-Lab2 qty=40
```

```
[AUDIT] action=ADD_EQUIPMENT | id=203 name=Arduino-Kit lab=ECE-Lab1 qty=25
[AUDIT] action=ADD_EQUIPMENT | id=204 name=Projector lab=CS-Lab2 qty=5
[AUDIT] action=REPORT | consolidated report generated
[AUDIT] action=ALLOCATE | id=202 qty=5 to=CS-Lab2 remaining=10
[AUDIT] action=ADD_EQUIPMENT | id=205 name=Multimeter lab=ECE-Lab1 qty=12
[AUDIT] action=MERGE | merged=1 conflicts=0
[AUDIT] action=MERGE | merged=0 conflicts=1
[AUDIT] action=SAVE | labguard_data.dat
[AUDIT] action=LOAD | labguard_data.dat
[AUDIT] action=REPORT | consolidated report generated
```

9. Analysis and Discussion

LabGuard-C demonstrates that a small set of deterministic rules - condition check, availability ratio, and a maintenance lock-out on allocation - is sufficient to turn a passive equipment register into an active decision-support tool, without any machine-learning component. The classification, search, and sort routines run in $O(n)$ or $O(n \log n)$ time over the equipment array, which is comfortably within the performance envelope of the array sizes used in this prototype ($MAX_EQUIPMENT = 200$).

The merge-conflict rule proved important in testing: without it, two labs independently numbering their equipment could silently overwrite each other's records the moment an ID collided. Flagging a mismatch on name or category instead of overwriting keeps the administrator in the loop for exactly the cases that matter. The main analytical limitation is that the 25% low-availability threshold and the maintenance-lockout rule are fixed constants chosen for plausibility rather than calibrated against real usage data from a live laboratory; this is stated explicitly rather than glossed over.

10. Conclusion

LabGuard-C shows that the core C constructs taught in this course - arrays, strings, searching, sorting, merging, functions, pointers, recursion, and file handling - are sufficient to build a coherent, explainable decision-support prototype for a genuinely useful real-world problem: keeping shared university laboratory equipment tracked, classified, safely allocated, and properly maintained across multiple labs. The system is deliberately scoped as a deterministic, rule-based prototype, with its limitations and future-enhancement path stated openly rather than omitted.

11. Individual Contribution of Group Members

Table 6 - Individual Contribution of Group Members

Member	Reg. No.	Contribution
[Student Name 1]	[Reg. No. 1]	System design, data structures, classification and allocation logic, pointer-based validation, recursion (availability projection, recursive search), report and documentation
[Student Name 2]	[Reg. No. 2]	Search/sort implementation, multi-lab merge and conflict detection, file handling (save/load, audit log), test-case design and execution

12. References

[1] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

[2] ISO/IEC 9899:2018 (C Standard) reference documentation, International Organization for Standardization, accessed 2026.

[3] GNU Project, "GCC, the GNU Compiler Collection - Online Documentation," Free Software Foundation, accessed 2026.

[4] Association for Educational Communications and Technology, guidance on laboratory and instructional-resource management in higher education, accessed 2026.

[5] United Nations, "The 17 Goals - Sustainable Development," Department of Economic and Social Affairs, accessed 2026.

13. One-Page Write-Up

Laboratories across a university campus each hold their own pool of shared equipment, and today that pool is tracked, if at all, in registers that record what exists but not what is safe to hand out right now. LabGuard-C was built to close exactly that gap using nothing beyond standard C: no external libraries, no database, no learned model - just arrays, strings, pointers, recursion, and a file on disk, arranged into a small decision-support pipeline.

The design follows five stages - Input, Analyse, Decide, Act, Prove. Input captures each equipment record (ID, name, category, quantity, available units, owning lab, condition, last maintenance date, and cumulative usage count) into a fixed-size array, with duplicate-ID checking and basic sanity validation enforced at the point of entry. Analyse derives, for every record, an availability ratio and a four-way classification - Available, Low Availability, Under Maintenance, or Critical - purely as a function of the stored fields, so the same inputs always produce the same, explainable output. Decide is where the interesting engineering sits: a pointer-based `validateAllocation` function refuses to hand out any unit whose condition is `Damaged` or `under Maintenance`, regardless of how urgently it is requested, and a merge routine combines equipment feeds submitted independently by different labs, flagging - rather than silently overwriting - any case where a shared ID disagrees on name or category between two labs. Act executes validated allocations directly through the pointer, and persists the whole equipment array to a binary file with a single `fwrite` call. Prove writes every add, allocation, merge, save, and load to a plain-text audit log, so any state change can be traced after the fact.

Two features were added specifically to meet the recursion requirement in a way that fits the problem rather than being bolted on: a recursive binary search over an ID-sorted array for fast lookup, and a recursive `projectedAvailability` function that walks forward day by day, subtracting a stated daily usage rate and clamping at zero, to answer a genuinely useful question - will this equipment run out before the semester's lab sessions are done? Both were exercised in testing (Section 7) with real, captured output rather than assumed results.

The result is a compact program - one source file, no dependencies beyond the standard library - that nonetheless does something a plain CRUD inventory program does not: it tells an administrator, in one consolidated report, which equipment is most heavily used, which labs are currently short, which items need servicing, and which need outright replacement. Its honestly stated limits - fixed array sizes, constants chosen for plausibility rather than calibrated data, and no authentication layer - are documented in this report rather than left implicit, consistent with the goal of a transparent, auditable classroom prototype rather than an overclaimed production system.

Appendix A - Complete Source Code

Compiler command used to verify the listing below compiles cleanly: gcc -Wall -Wextra -o labguard labguard.c (0 errors, 0 warnings).

```
/* =====
LabGuard-C : Smart University Laboratory Equipment Monitoring and
Allocation System
Course      : C Programming (Course Outcomes C02, C03, C05)
Team       : [Student Name 1] ([Reg. No. 1])
            [Student Name 2] ([Reg. No. 2])
-----
Rule-based / deterministic decision support for tracking, classifying,
allocating, and maintaining laboratory equipment across multiple
university laboratories.
===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EQUIPMENT  200
#define MAX_LABS       20
#define NAME_LEN       40
#define CATEGORY_LEN   30
#define LAB_LEN        30
#define COND_LEN       20
#define DATE_LEN       12
#define DATA_FILE     "labguard_data.dat"
#define AUDIT_FILE     "labguard_audit.log"

/* -----
1. DATA STRUCTURES
----- */
typedef struct {
    int id;
    char name[NAME_LEN];
    char category[CATEGORY_LEN];
    int quantity; /* total units owned */
    int availableUnits; /* units currently free to allocate */
    char lab[LAB_LEN]; /* laboratory / department the item sits in */
    char condition[COND_LEN]; /* Good / Fair / Damaged / Maintenance */
    char maintenanceDate[DATE_LEN]; /* DD-MM-YYYY, last service date */
    int usageCount; /* cumulative number of times allocated */
} Equipment;

typedef struct {
    char lab[LAB_LEN];
    int activeUsers; /* students/staff typically served */
    int pendingRequests; /* outstanding allocation requests */
} LabProfile;

static Equipment equipment[MAX_EQUIPMENT];
static int equipmentCount = 0;

static LabProfile labProfiles[MAX_LABS];
static int labCount = 0;

/* -----
Utility: audit trail (file-handling requirement)
----- */
void logAudit(const char *action, const char *detail) {
    FILE *fp = fopen(AUDIT_FILE, "a");
    if (!fp) return;
```

```

    fprintf(fp, "[AUDIT] action=%s | %s\n", action, detail);
    fclose(fp);
}

/* -----
2. CORE REQUIREMENTS: operators, decision-making, loops, arrays, strings
----- */

int findEquipmentIndexById(int id) {
    for (int i = 0; i < equipmentCount; i++) {
        if (equipment[i].id == id) return i;
    }
    return -1;
}

int isDuplicateId(int id) {
    return findEquipmentIndexById(id) != -1;
}

int addEquipment(Equipment e) {
    if (equipmentCount >= MAX_EQUIPMENT) return 0;
    if (isDuplicateId(e.id)) return 0;
    if (e.quantity < 0 || e.availableUnits < 0 || e.usageCount < 0) return 0;
    if (e.availableUnits > e.quantity) return 0;
    equipment[equipmentCount++] = e;
    char buf[100];
    snprintf(buf, sizeof(buf), "id=%d name=%s lab=%s qty=%d",
             e.id, e.name, e.lab, e.quantity);
    logAudit("ADD_EQUIPMENT", buf);
    return 1;
}

int updateCondition(int id, const char *newCondition, const char *newDate) {
    int idx = findEquipmentIndexById(id);
    if (idx == -1) return 0;
    strncpy(equipment[idx].condition, newCondition, COND_LEN - 1);
    equipment[idx].condition[COND_LEN - 1] = '\0';
    strncpy(equipment[idx].maintenanceDate, newDate, DATE_LEN - 1);
    equipment[idx].maintenanceDate[DATE_LEN - 1] = '\0';
    char buf[80];
    snprintf(buf, sizeof(buf), "id=%d condition=%s date=%s", id, newCondition, newDate);
    logAudit("UPDATE_CONDITION", buf);
    return 1;
}

/* -----
3. SEARCHING: linear substring search, iterative + recursive binary search
----- */

void searchByName(const char *keyword) {
    printf("\nSearch results for \"%s\":\n", keyword);
    int found = 0;
    for (int i = 0; i < equipmentCount; i++) {
        if (strstr(equipment[i].name, keyword) != NULL) {
            printf(" [%d] %s | Lab=%s | Category=%s | Avail=%d/%d\n",
                 equipment[i].id, equipment[i].name, equipment[i].lab,
                 equipment[i].category, equipment[i].availableUnits,
                 equipment[i].quantity);
            found = 1;
        }
    }
    if (!found) printf(" No matching equipment found.\n");
}

void searchByCategory(const char *category) {
    printf("\nEquipment in category \"%s\":\n", category);
    int found = 0;
    for (int i = 0; i < equipmentCount; i++) {

```

```

        if (strcmp(equipment[i].category, category) == 0) {
            printf(" [%d] %s @ %s\n", equipment[i].id, equipment[i].name, equipment[i].lab);
            found = 1;
        }
    }
    if (!found) printf(" No equipment found in this category.\n");
}

int compareById(const void *a, const void *b) {
    return ((Equipment *)a)->id - ((Equipment *)b)->id;
}

/* Recursive binary search on an id-sorted array (recursion requirement #1) */
int binarySearchByIdRecursive(Equipment *sorted, int lo, int hi, int id) {
    if (lo > hi) return -1;
    int mid = (lo + hi) / 2;
    if (sorted[mid].id == id) return mid;
    if (sorted[mid].id < id) return binarySearchByIdRecursive(sorted, mid + 1, hi, id);
    return binarySearchByIdRecursive(sorted, lo, mid - 1, id);
}

/* -----
4. SORTING: usage-frequency descending, availability ascending
----- */
void sortByUsageDesc(Equipment arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int maxIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j].usageCount > arr[maxIdx].usageCount) maxIdx = j;
        }
        if (maxIdx != i) {
            Equipment tmp = arr[i];
            arr[i] = arr[maxIdx];
            arr[maxIdx] = tmp;
        }
    }
}

void sortByAvailabilityAsc(Equipment arr[], int n) {
    for (int i = 1; i < n; i++) {
        Equipment key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j].availableUnits > key.availableUnits) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

/* -----
5. DECISION INTELLIGENCE: availability ratio + condition-based classifier
----- */
int computeAvailabilityRatio(Equipment e) {
    /* Percentage of total stock currently available, 0-100 */
    if (e.quantity == 0) return 0;
    return (e.availableUnits * 100) / e.quantity;
}

const char *classifyEquipment(Equipment e) {
    if (strcmp(e.condition, "Damaged") == 0 || strcmp(e.condition, "Maintenance") == 0)
        return "UNDER MAINTENANCE";
    if (e.availableUnits == 0)
        return "CRITICAL";
    if (computeAvailabilityRatio(e) < 25)
        return "LOW AVAILABILITY";
}

```



```

    return "AVAILABLE";
}

/* -----
6. LAB RISK / SHORTAGE INDICATOR
----- */
int labHasShortage(const char *labName) {
    for (int i = 0; i < equipmentCount; i++) {
        if (strcmp(equipment[i].lab, labName) == 0) {
            const char *cls = classifyEquipment(equipment[i]);
            if (strcmp(cls, "CRITICAL") == 0 || strcmp(cls, "LOW AVAILABILITY") == 0)
                return 1;
        }
    }
    return 0;
}

/* -----
7. RECURSION: projected-depletion analysis (recursion requirement #2)
Models how many units remain available after N days of use at a
given average daily consumption/borrow rate, clamped at zero.
----- */
int projectedAvailability(int availableUnits, int dailyUsageRate, int daysAhead) {
    if (daysAhead == 0) return availableUnits;
    int remaining = availableUnits - dailyUsageRate;
    if (remaining < 0) remaining = 0;
    return projectedAvailability(remaining, dailyUsageRate, daysAhead - 1);
}

/* -----
8. POINTERS: pointer-based safe allocation
----- */
int validateAllocation(Equipment *e, int requestedQty) {
    if (e == NULL) return 0;
    if (requestedQty <= 0 || requestedQty > e->availableUnits) return 0;
    if (strcmp(e->condition, "Damaged") == 0 || strcmp(e->condition, "Maintenance") == 0)
        return 0; /* cannot allocate equipment under maintenance */
    return 1;
}

int allocateEquipment(Equipment *e, int requestedQty, const char *requestingLab) {
    if (!validateAllocation(e, requestedQty)) return 0;
    e->availableUnits -= requestedQty;
    e->usageCount += requestedQty;
    char buf[100];
    snprintf(buf, sizeof(buf), "id=%d qty=%d to=%s remaining=%d",
             e->id, requestedQty, requestingLab, e->availableUnits);
    logAudit("ALLOCATE", buf);
    return 1;
}

/* -----
9. MULTI-LAB MERGE + CONFLICT DETECTION
----- */
typedef struct {
    int idA, idB;
    char reason[100];
} MergeConflict;

int mergeLabData(Equipment incoming[], int incomingCount,
                 MergeConflict conflicts[], int *conflictCount) {
    int merged = 0;
    *conflictCount = 0;
    for (int i = 0; i < incomingCount; i++) {
        int idx = findEquipmentIndexById(incoming[i].id);
        if (idx == -1) {

```

```

        if (addEquipment(incoming[i])) merged++;
    } else {
        if (strcmp(equipment[idx].name, incoming[i].name) != 0 ||
            strcmp(equipment[idx].category, incoming[i].category) != 0) {
            conflicts[*conflictCount].idA = equipment[idx].id;
            conflicts[*conflictCount].idB = incoming[i].id;
            snprintf(conflicts[*conflictCount].reason, 100,
                "Identity mismatch for id %d (name/category differ across labs)",
                incoming[i].id);
            (*conflictCount)++;
        } else {
            equipment[idx].quantity += incoming[i].quantity;
            equipment[idx].availableUnits += incoming[i].availableUnits;
            merged++;
        }
    }
}
char buf[64];
snprintf(buf, sizeof(buf), "merged=%d conflicts=%d", merged, *conflictCount);
logAudit("MERGE", buf);
return merged;
}

/* -----
10. FILE HANDLING: persistent storage of equipment records
----- */
int saveToFile(void) {
    FILE *fp = fopen(DATA_FILE, "wb");
    if (!fp) return 0;
    fwrite(&equipmentCount, sizeof(int), 1, fp);
    fwrite(equipment, sizeof(Equipment), equipmentCount, fp);
    fclose(fp);
    logAudit("SAVE", DATA_FILE);
    return 1;
}

int loadFromFile(void) {
    FILE *fp = fopen(DATA_FILE, "rb");
    if (!fp) return 0;
    fread(&equipmentCount, sizeof(int), 1, fp);
    fread(equipment, sizeof(Equipment), equipmentCount, fp);
    fclose(fp);
    logAudit("LOAD", DATA_FILE);
    return 1;
}

/* -----
11. CONSOLIDATED REPORT
----- */
void generateConsolidatedReport(void) {
    printf("\n===== LabGuard-C CONSOLIDATED REPORT =====\n");
    printf("%-4s %-14s %-12s %-10s %6s %6s %-16s\n",
        "ID", "Name", "Lab", "Category", "Avail", "Qty", "Status");
    int totalAvailable = 0, mostUsedIdx = -1, criticalCount = 0, maintCount = 0;
    for (int i = 0; i < equipmentCount; i++) {
        Equipment e = equipment[i];
        printf("%-4d %-14s %-12s %-10s %6d %6d %-16s\n",
            e.id, e.name, e.lab, e.category, e.availableUnits, e.quantity,
            classifyEquipment(e));
        totalAvailable += e.availableUnits;
        if (mostUsedIdx == -1 || e.usageCount > equipment[mostUsedIdx].usageCount)
            mostUsedIdx = i;
        if (strcmp(classifyEquipment(e), "CRITICAL") == 0) criticalCount++;
        if (strcmp(classifyEquipment(e), "UNDER MAINTENANCE") == 0) maintCount++;
    }
    printf("-----\n");
    if (mostUsedIdx != -1)

```

```

        printf("Most frequently used equipment : %s (usage=%d)\n",
               equipment[mostUsedIdx].name, equipment[mostUsedIdx].usageCount);
    printf("Total available units (all labs): %d\n", totalAvailable);
    printf("Equipment requiring maintenance : %d\n", maintCount);
    printf("Equipment requiring replacement/procurement (CRITICAL): %d\n", criticalCount);
    printf("Labs with shortages: ");
    int any = 0;
    for (int i = 0; i < labCount; i++) {
        if (labHasShortage(labProfiles[i].lab)) {
            printf("%s%s", any ? ", " : "", labProfiles[i].lab);
            any = 1;
        }
    }
    if (!any) printf("None");
    printf("\n=====\\n");
    logAudit("REPORT", "consolidated report generated");
}

/* -----
MAIN: demonstration driver (menu-driven in the full submission build)
----- */
int main(void) {
    strcpy(labProfiles[labCount].lab, "ECE-Lab1");
    labProfiles[labCount].activeUsers = 60;
    labProfiles[labCount].pendingRequests = 4;
    labCount++;

    strcpy(labProfiles[labCount].lab, "CS-Lab2");
    labProfiles[labCount].activeUsers = 90;
    labProfiles[labCount].pendingRequests = 2;
    labCount++;

    Equipment e1 = {201, "Oscilloscope",    "Measuring",    10, 2,    "ECE-Lab1", "Good",    "10-06-2026",
45};
    Equipment e2 = {202, "Desktop-PC",      "Computer",     40, 15,    "CS-Lab2", "Good",    "01-07-2026",
320};
    Equipment e3 = {203, "Arduino-Kit",     "Electronic",   25, 0,    "ECE-Lab1", "Maintenance", "15-05-2026",
210};
    Equipment e4 = {204, "Projector",       "AV",           5, 1,    "CS-Lab2", "Fair",    "20-04-2026",
60};

    addEquipment(e1);
    addEquipment(e2);
    addEquipment(e3);
    addEquipment(e4);
    printf("Duplicate add attempt result: %d (expect 0)\\n", addEquipment(e1));

    searchByName("Kit");
    searchByCategory("Measuring");

    Equipment sortedById[MAX_EQUIPMENT];
    memcpy(sortedById, equipment, sizeof(Equipment) * equipmentCount);
    qsort(sortedById, equipmentCount, sizeof(Equipment), compareById);
    int pos = binarySearchByIdRecursive(sortedById, 0, equipmentCount - 1, 203);
    printf("\\nRecursive binary search for id=203 -> array index %d (name=%s)\\n",
        pos, pos != -1 ? sortedById[pos].name : "N/A");

    sortByUsageDesc(equipment, equipmentCount);
    generateConsolidatedReport();

    int idx = findEquipmentIndexById(202);
    printf("\\nAllocation attempt: 5 x Desktop-PC to CS-Lab2 -> result=%d\\n",
        allocateEquipment(&equipment[idx], 5, "CS-Lab2"));
    printf("Allocation attempt: 3 x Arduino-Kit (under maintenance) -> result=%d (expect 0)\\n",
        allocateEquipment(&equipment[findEquipmentIndexById(203)], 3, "ECE-Lab1"));

```

```

int projected = projectedAvailability(equipment[idx].availableUnits, 3, 4);
printf("\nProjected Desktop-PC availability after 4 days at 3 units/day usage: %d\n", projected);

Equipment incoming[1] = {{205, "Multimeter", "Measuring", 12, 12, "ECE-Lab1", "Good", "05-07-2026",
8}};
MergeConflict conflicts[10];
int conflictCount = 0;
mergeLabData(incoming, 1, conflicts, &conflictCount);
printf("\nMerge from ECE-Lab1 feed -> conflicts detected: %d\n", conflictCount);

Equipment conflictIncoming[1] = {{204, "HD-Projector", "AV", 3, 3, "CS-Lab2", "Good", "01-01-2026",
0}};
mergeLabData(conflictIncoming, 1, conflicts, &conflictCount);
printf("Merge with mismatched id=204 -> conflicts detected: %d\n", conflictCount);
if (conflictCount > 0) printf(" Reason: %s\n", conflicts[0].reason);

saveToFile();
equipmentCount = 0;
loadFromFile();
printf("\nReloaded equipment count from file: %d\n", equipmentCount);

generateConsolidatedReport();
return 0;
}

```

Appendix B - Sample Data

id	name	category	qty	avail	lab	condition	usageCount
201	Oscilloscope	Measuring	10	2	ECE-Lab1	Good	45
202	Desktop-PC	Computer	40	15	CS-Lab2	Good	320
203	Arduino-Kit	Electronic	25	0	ECE-Lab1	Maintenance	210
204	Projector	AV	5	1	CS-Lab2	Fair	60
205	Multimeter	Measuring	12	12	ECE-Lab1	Good	8

OUTPUT:

```

SMART UNIVERSITY LABORATORY EQUIPMENT MONITORING & ALLOCATION SYSTEM
Loaded 0 equipment record(s) from 'equipment_data.txt'.

=====
1. Add Equipment
2. Update Equipment
3. Display All Equipment
4. Search Equipment (ID / Category / Lab / Condition / Availability)
5. Sort Equipment (ID / Category / Usage / Availability)
6. Merge Records from Another Laboratory File
7. Analyse Equipment Status (Available/Low/Maintenance/Critical)
8. Allocate Equipment to a Requesting Lab/User
9. Generate Consolidated Report
10. Save Records to File
0. Exit

Enter your choice: 1
Enter Equipment ID: 101
Enter Equipment Name: computer
Enter Category (e.g. Computer/Sensor/Kit/Instrument): computer
Enter Laboratory Name: computer lab
Enter Condition (Good/Damaged/Under Repair): good
Enter Last Maintenance Date (DD-MM-YYYY): 15-08-2026
Enter Total Quantity: 20
Enter Available Units: 15
Enter Current Usage Count: 120
Equipment 'computer' added successfully with status [Available].

=====

```

```

Enter your choice: 1
Enter Equipment ID: 103
Enter Equipment Name: arduino kit
Enter Category (e.g. Computer/Sensor/Kit/Instrument): kit
Enter Laboratory Name: embedded lab
Enter Condition (Good/Damaged/Under Repair): under repair
Enter Last Maintenance Date (DD-MM-YYYY): 10-07-2026
Enter Total Quantity: 15
Enter Available Units: 0
Enter Current Usage Count: 300
Equipment 'arduino kit' added successfully with status [Under Maintenance].

```

```

=====
Enter your choice: 1
Enter Equipment ID: temperature sensor
Invalid input. Please enter a number: sensor
Invalid input. Please enter a number: iot lab
Invalid input. Please enter a number: 102
Enter Equipment Name: temperature sensor
Enter Category (e.g. Computer/Sensor/Kit/Instrument): sensor
Enter Laboratory Name: iot lab
Enter Condition (Good/Damaged/Under Repair): good
Enter Last Maintenance Date (DD-MM-YYYY): 20-08-2026
Enter Total Quantity: 10
Enter Available Units: 2
Enter Current Usage Count: 450
Equipment 'temperature sensor' added successfully with status [Low Availability].

```

```
-----  
Enter your choice: 3
```

```
===== ALL EQUIPMENT RECORDS (3) =====  
-----
```

```
ID: 101   Name: computer           Category: computer  
Lab: computer lab Condition: good      Maint.Date: 15-08-2026  
Qty: 20   Available: 15   Usage: 120   Status: Available  
-----
```

```
ID: 102   Name: temperature sensor Category: sensor  
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026  
Qty: 10   Available: 2     Usage: 450   Status: Low Availability  
-----
```

```
ID: 103   Name: arduino kit        Category: kit  
Lab: embedded lab Condition: under repair Maint.Date: 10-07-2026  
Qty: 15   Available: 0     Usage: 300   Status: Under Maintenance  
-----
```

```
-----  
Enter your choice: 4
```

```
Search by: 1.ID(Recursive Binary) 2.Category 3.Lab 4.Condition 5.Availability  
Choice: 1
```

```
Enter Equipment ID: 102
```

```
Record found (recursive binary search):  
-----
```

```
ID: 102   Name: temperature sensor Category: sensor  
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026  
Qty: 10   Available: 2     Usage: 450   Status: Low Availability  
-----
```

```
-----  
Enter your choice: 4
```

```
Search by: 1.ID(Recursive Binary) 2.Category 3.Lab 4.Condition 5.Availability  
Choice: 2
```

```
Enter Category: computer  
-----
```

```
ID: 101   Name: computer           Category: computer  
Lab: computer lab Condition: good      Maint.Date: 15-08-2026  
Qty: 20   Available: 15   Usage: 120   Status: Available  
1 record(s) found.
```

```
-----  
Enter your choice: 4
```

```
Search by: 1.ID(Recursive Binary) 2.Category 3.Lab 4.Condition 5.Availability  
Choice: 3
```

```
Enter Lab Name: iot lab  
-----
```

```
ID: 102   Name: temperature sensor Category: sensor  
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026  
Qty: 10   Available: 2     Usage: 450   Status: Low Availability  
1 record(s) found.
```

```

Enter your choice: 4
Search by: 1.ID(Recursive Binary) 2.Category 3.Lab 4.Condition 5.Availability
Choice: 4
Enter Condition: good
-----
ID: 101   Name: computer       Category: computer
Lab: computer lab Condition: good      Maint.Date: 15-08-2026
Qty: 20   Available: 15   Usage: 120   Status: Available
-----
ID: 102   Name: temperature sensor Category: sensor
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026
Qty: 10   Available: 2     Usage: 450   Status: Low Availability
2 record(s) found.

```

```

Enter your choice: 4
Search by: 1.ID(Recursive Binary) 2.Category 3.Lab 4.Condition 5.Availability
Choice: 5
-----
ID: 101   Name: computer       Category: computer
Lab: computer lab Condition: good      Maint.Date: 15-08-2026
Qty: 20   Available: 15   Usage: 120   Status: Available
-----
ID: 102   Name: temperature sensor Category: sensor
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026
Qty: 10   Available: 2     Usage: 450   Status: Low Availability
2 record(s) currently available.

```

```

Enter your choice: 5
Sort by: 1.ID 2.Category(A-Z) 3.Usage(High-Low) 4.Availability(High-Low)
Choice: 1
Sorted by ID.

===== ALL EQUIPMENT RECORDS (3) =====
-----
ID: 101   Name: computer       Category: computer
Lab: computer lab Condition: good      Maint.Date: 15-08-2026
Qty: 20   Available: 15   Usage: 120   Status: Available
-----
ID: 102   Name: temperature sensor Category: sensor
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026
Qty: 10   Available: 2     Usage: 450   Status: Low Availability
-----
ID: 103   Name: arduino kit     Category: kit
Lab: embedded lab Condition: under repair Maint.Date: 10-07-2026
Qty: 15   Available: 0     Usage: 300   Status: Under Maintenance

```



```
Enter your choice: 5
Sort by: 1.ID 2.Category(A-Z) 3.Usage(High-Low) 4.Availability(High-Low)
Choice: 3
Sorted by Usage (descending).
```

```
===== ALL EQUIPMENT RECORDS (3) =====
-----
ID: 102  Name: temperature sensor Category: sensor
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026
Qty: 10  Available: 2  Usage: 450  Status: Low Availability
-----
ID: 103  Name: arduino kit      Category: kit
Lab: embedded lab Condition: under repair Maint.Date: 10-07-2026
Qty: 15  Available: 0  Usage: 300  Status: Under Maintenance
-----
ID: 101  Name: computer      Category: computer
Lab: computer lab Condition: good      Maint.Date: 15-08-2026
Qty: 20  Available: 15  Usage: 120  Status: Available
```

```
Enter your choice: 5
Sort by: 1.ID 2.Category(A-Z) 3.Usage(High-Low) 4.Availability(High-Low)
Choice: 2
Sorted by Category.
```

```
===== ALL EQUIPMENT RECORDS (3) =====
-----
ID: 101  Name: computer      Category: computer
Lab: computer lab Condition: good      Maint.Date: 15-08-2026
Qty: 20  Available: 15  Usage: 120  Status: Available
-----
ID: 103  Name: arduino kit      Category: kit
Lab: embedded lab Condition: under repair Maint.Date: 10-07-2026
Qty: 15  Available: 0  Usage: 300  Status: Under Maintenance
-----
ID: 102  Name: temperature sensor Category: sensor
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026
Qty: 10  Available: 2  Usage: 450  Status: Low Availability
-----
```

```
Enter your choice: 5
Sort by: 1.ID 2.Category(A-Z) 3.Usage(High-Low) 4.Availability(High-Low)
Choice: 4
Sorted by Availability (descending).
```

```
===== ALL EQUIPMENT RECORDS (3) =====
-----
ID: 101  Name: computer      Category: computer
Lab: computer lab Condition: good      Maint.Date: 15-08-2026
Qty: 20  Available: 15  Usage: 120  Status: Available
-----
ID: 102  Name: temperature sensor Category: sensor
Lab: iot lab      Condition: good      Maint.Date: 20-08-2026
Qty: 10  Available: 2  Usage: 450  Status: Low Availability
-----
ID: 103  Name: arduino kit      Category: kit
Lab: embedded lab Condition: under repair Maint.Date: 10-07-2026
Qty: 15  Available: 0  Usage: 300  Status: Under Maintenance
```


Enter your choice: 7

```
===== EQUIPMENT STATUS ANALYSIS =====  
[Available      ] ID:101  computer      (Lab: computer lab | Avail: 15/20)  
[Low Availability] ID:102  temperature sensor (Lab: iot lab    | Avail: 2/10)  
[Under Maintenance] ID:103  arduino kit    (Lab: embedded lab | Avail: 0/15)  
-----  
Summary => Available: 1 | Low Availability: 1 | Under Maintenance: 1 | Critical: 0
```

Enter your choice: 8
Enter Equipment ID to allocate: 101
Enter number of units requested: 2
Allocated 2 unit(s) of 'computer'. Remaining available: 13. New status: [Available]

Enter your choice: 9

Report generated and saved to 'lab_report.txt'.

```
----- REPORT PREVIEW -----  
=====
```

CONSOLIDATED LABORATORY EQUIPMENT REPORT

```
=====
```

- 1) MOST FREQUENTLY USED EQUIPMENT:
temperature sensor (ID:102) - Used 450 times, Lab: iot lab
- 2) LABORATORIES WITH SHORTAGES:
- iot lab (e.g. temperature sensor is Low Availability)
- 3) EQUIPMENT REQUIRING MAINTENANCE:
- ID:103 arduino kit (Lab: embedded lab, Last Maint: 10-07-2026)
- 4) TOTAL AVAILABLE EQUIPMENT UNITS: 15
- 5) EQUIPMENT REQUIRING IMMEDIATE REPLACEMENT/PROCUREMENT:
None.

```
1. Add Equipment
2. Update Equipment
3. Display All Equipment
4. Search Equipment (ID / Category / Lab / Condition / Availability)
5. Sort Equipment (ID / Category / Usage / Availability)
6. Merge Records from Another Laboratory File
7. Analyse Equipment Status (Available/Low/Maintenance/Critical)
8. Allocate Equipment to a Requesting Lab/User
9. Generate Consolidated Report
10. Save Records to File
0. Exit
```

Enter your choice: 10

Data saved to 'equipment_data.txt'.

```
1. Add Equipment
2. Update Equipment
3. Display All Equipment
4. Search Equipment (ID / Category / Lab / Condition / Availability)
5. Sort Equipment (ID / Category / Usage / Availability)
6. Merge Records from Another Laboratory File
7. Analyse Equipment Status (Available/Low/Maintenance/Critical)
8. Allocate Equipment to a Requesting Lab/User
9. Generate Consolidated Report
10. Save Records to File
0. Exit
```

Enter your choice: 0

Data auto-saved. Exiting. Goodbye!

...Program finished with exit code 0
Press ENTER to exit console.

